

Análisis y Diseño con Patrones

Práctica 2: Patrones de Diseño y de Asignación de Responsabilidades (evaluación sobre 50 puntos)

Pregunta 1 (15 puntos)

Enunciado

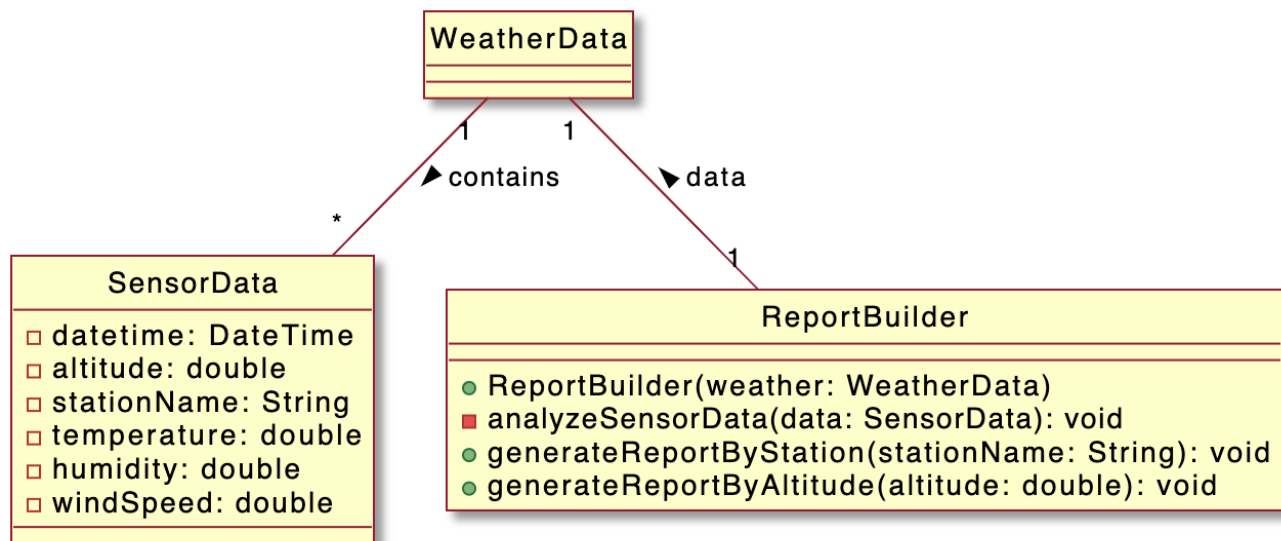
Un sistema meteorológico recoge datos de varios sensores situados en diferentes ubicaciones geográficas. Cada sensor registra datos sobre la temperatura, la humedad y la velocidad del viento. El sistema guarda los datos de los sensores en un repositorio central donde se puede acceder para el análisis y la presentación de informes.

El diseño actual del sistema tiene una clase *WeatherData* que mantiene una lista de datos de los sensores. Cada entrada de datos de los sensores contiene información como la altitud, el nombre de la estación, la temperatura, la humedad y la velocidad del viento. A menudo, los analistas necesitan agregar estos datos para generar informes o realizar análisis.

La clase *ReportBuilder* es responsable de generar informes basados en los datos de los sensores. Puede generar un informe seleccionando los datos de los sensores por el nombre de la estación, o puede hacer un informe seleccionando los datos de los sensores donde la altitud es superior a la reportada. Para generar los informes, se tendría que llamar al método *analysisSensorData* con todos los datos del sensor necesarios.

Cómo es habitual, tendríamos que tener el acoplamiento mínimo entre *ReportBuilder*, *WeatherData* y *SensorData*. Los detalles internos de *WeatherData* y *SensorData* no se tienen que exponer a *ReportBuilder* mientras queremos proporcionar diferentes maneras de recorrer los datos de los sensores.

A continuación disponéis del diagrama de clases de este sistema:



Restricciones de integridad

- SensorData se identifica por datetime y stationName.

Se pide:

- (3 puntos) Justifica qué patrón de diseño se tiene que aplicar para generar los informes por el nombre de la estación y también por la altitud.
- (5 puntos) Muestra el diagrama de clases con las operaciones necesarias para generar los informes por el nombre de la estación y también por la altitud, recorriendo los datos disponibles en la clase *SensorData* dependiendo del tipo de informe.
- (7 puntos) Muestra el pseudocódigo completo de las clases que son necesarias para implementar las operaciones *ReportBuilder::generateReportByStation* y *ReportBuilder::generateReportByAltitude*.

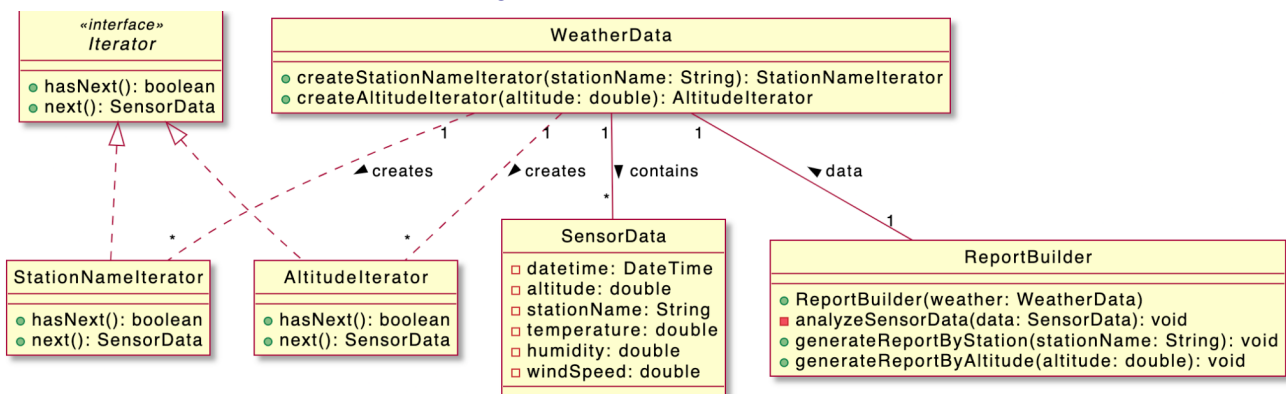
Solución

- Usaremos el **patrón Iterador** para definir dos iteradores diferentes, para filtrar los datos de la clase *SensorData* a partir del nombre de la estación y de la altitud. Así, podemos encapsular la lógica del recorrido y filtraje en los iteradores, y por tanto, los detalles internos de *WeatherData* y *SensorData* no se muestran en la clase *ReportBuilder* proporcionando una implementación más simple. La clase *ReportBuilder* usará los iteradores para acceder a los objetos de la clase *SensorData*, el que permite minimizar el acoplamiento.

Los iteradores que definimos son:

- *StationNameIterator*: Itera sobre los objetos de la clase *SensorData* y obvia los objetos que no correspondan al nombre de la estación.
- *AltitudeIterator*: Itera sobre los objetos de la clase *SensorData* y obvia los objetos que correspondan a una altitud menor que la especificada por un threshold.

b) A continuación disponéis el diagrama de clases resultante de aplicar el patrón iterador:



c) A continuación disponéis del pseudocódigo de las clases involucradas en las operaciones que se piden:

```

// Interface for the Iterators
interface Iterator {
    boolean hasNext();
    SensorData next();
}

// Iterator that filters by station name
class StationNameIterator implements Iterator {
    private List<SensorData> sensorData;
    private int currentIndex = 0;
    private String stationName;

    StationNameIterator(List<SensorData> sensorData, String stationName) {
        this.sensorData = sensorData;
        this.stationName = stationName;
    }

    public boolean hasNext() {
        while (currentIndex < sensorData.size()) {
            SensorData data = sensorData.get(currentIndex);
            if (data.getStationName().equals(stationName)) {
                return true;
            }
            currentIndex++;
        }
        return false;
    }

    public SensorData next() {
        SensorData data = sensorData.get(currentIndex);
        currentIndex++;
        return data;
    }
}

```

```

        return true;
    }
    currentIndex++;
}
return false;
}

public SensorData next() {
    if (!hasNext()) {
        throw new NoSuchElementException();
    }
    return sensorData.get(currentIndex++);
}
}

// Iterator that filters by altitude
class AltitudeIterator implements Iterator {
    private List<SensorData> sensorData;
    private int currentIndex = 0;
    private double altitude;

    AltitudeIterator(List<SensorData> sensorData, double altitude) {
        this.sensorData = sensorData;
        this.altitude = altitude;
    }

    public boolean hasNext() {
        while (currentIndex < sensorData.size()) {
            SensorData data = sensorData.get(currentIndex);
            if (data.getAltitude() > altitude) {
                return true;
            }
            currentIndex++;
        }
        return false;
    }

    public SensorData next() {
        if (!hasNext()) {
            throw new NoSuchElementException();
        }
        return sensorData.get(currentIndex++);
    }
}

```

```

    }
}

// WeatherData class creates the appropriate iterator
class WeatherData {
    private List<SensorData> sensorData;

    Iterator createStationNameIterator(String stationName) {
        return new StationNameIterator(sensorData, stationName);
    }

    Iterator createAltitudeIterator(double altitude) {
        return new AltitudeIterator(sensorData, altitude);
    }
}

// ReportBuilder uses the provided iterator to generate the report
class ReportBuilder {
    private WeatherData weatherData;

    ReportBuilder(WeatherData weatherData) {
        this.weatherData = weatherData;
    }

    void generateReportByStation(String stationName) {
        generateReport(weatherData.createStationNameIterator(stationName));
    }

    void generateReportByAltitude(double altitude) {
        generateReport(weatherData.createAltitudeIterator(altitude));
    }

    void generateReport(Iterator iterator) {
        while (iterator.hasNext()) {
            SensorData data = iterator.next();
            analyzeSensorData(data);
        }
    }
}

```

Pregunta 2 (10 puntos)

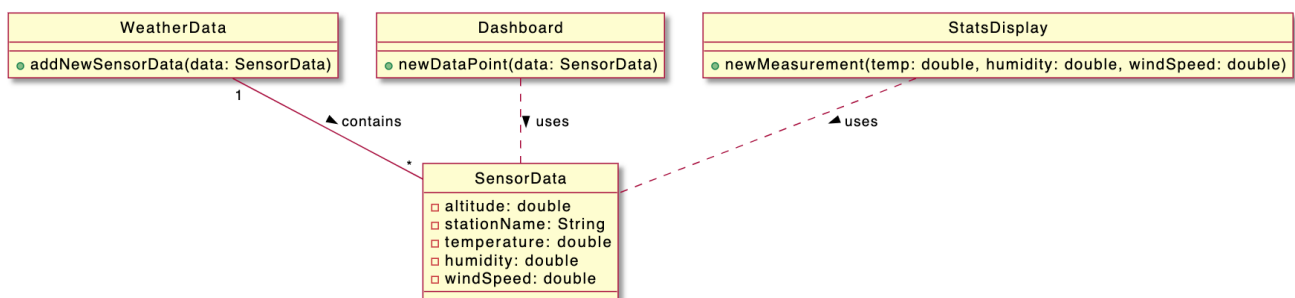
Enunciado

En el sistema meteorológico del ejercicio anterior se añaden dos nuevos componentes: un dashboard y un visor de estadísticas. El dashboard es una interfaz de usuario gráfica que muestra los datos meteorológicos más recientes de las estaciones meteorológicas. Por otro lado, el visor de estadísticas es un componente que analiza los datos meteorológicos y muestra los valores mínimos, máximos y medianos de cada medida.

Podéis suponer que las clases *Dashboard* y *StatsDisplay* ya están implementadas y que tienen métodos denominados *newDataPoint* y *newMeasurement*, respectivamente. Estos métodos se tendrán que llamar siempre que haya datos nuevos, es decir, cuando se invoque la operación *addNewSensorData* de la clase *WeatherData*.

Cómo es habitual, queremos añadir estos dos nuevos componentes con el mínimo acoplamiento al sistema existente.

A continuación disponéis del diagrama de clases inicial de este sistema:

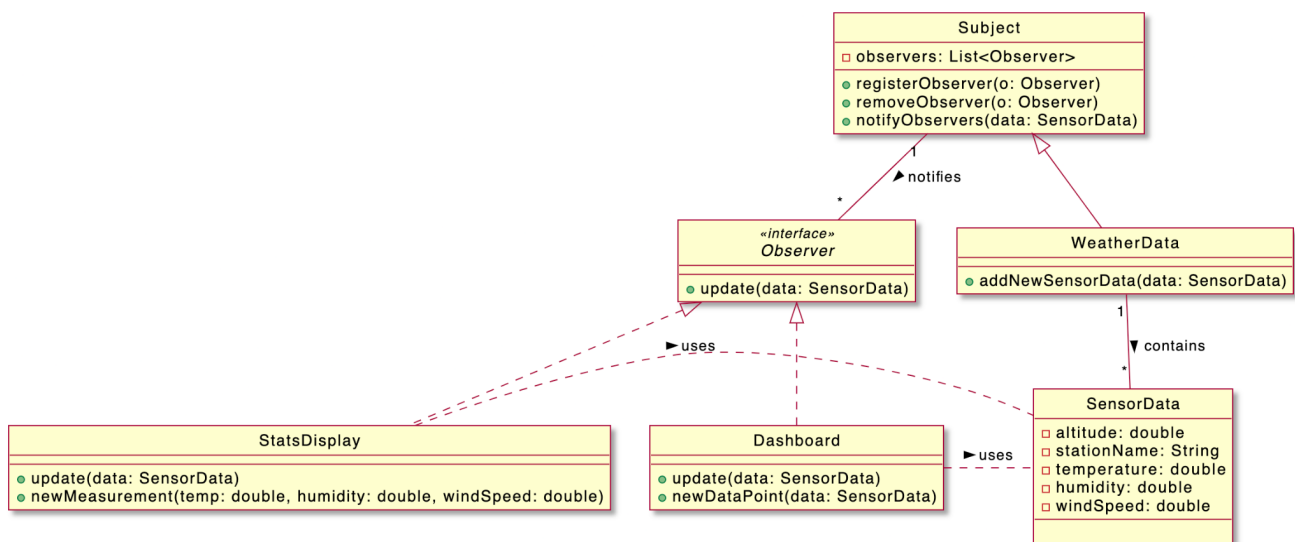


Se pide:

- (3 puntos) Justifica qué patrón de diseño se tiene que aplicar para integrar las clases *Dashboard* and *StatsDisplay* al sistema meteorológico.
- (3 puntos) Muestra el diagrama de clases con las operaciones necesarias para completar el diseño.
- (4 puntos) Muestra el pseudocódigo completo de las clases con las operaciones o el diagrama de secuencia que muestre cómo las operaciones `Dashboard::newDataPoint(data: SensorData)` y `StatsDisplay::newMeasurement(data: SensorData)` son invocadas.

Solución

- a) Para integrar las clases *Dashboard* and *StatsDisplay* con el mínimo acoplamiento, y para gestionar que cuando nuevos datos de un sensor son añadidos en la clase *WeatherData*, el dashboard y el visor de estadísticas se actualicen, usaremos el **patrón Observador**. Este patrón define una relación tal que cuando un objeto cambia su estado, todos sus dependientes son notificados y actualizados automáticamente. La clase *WeatherData* será el subject, y las clases *Dashboard* y *StatsDisplay* serán los observadores. Cuando la clase *WeatherData* reciba nuevos datos, notificará a sus observadores que tiene nuevos datos disponibles para que ellos se actualicen.
- b) A continuación disponéis del diagrama de clases resultante con la aplicación del patrón observador:



- c) A continuación disponéis del pseudocódigo de las clases involucradas en las operaciones que se piden:

```

interface Observer {
    void update(SensorData data);
}

class Subject {
    private List<Observer> observers;

    Subject() {
        observers = new ArrayList<>();
    }

    void registerObserver(Observer o) {
        observers.add(o);
    }
}
    
```

```

void removeObserver(Observer o) {
    observers.remove(o);
}

void notifyObservers(SensorData data) {
    for (Observer observer : observers) {
        observer.update(data);
    }
}

class SensorData {
    private double altitude;
    private String stationName;
    private double temperature;
    private double humidity;
    private double windSpeed;

    // Accessor methods for SensorData fields
    // ...
}

class WeatherData extends Subject {
    private List<SensorData> sensorData;

    void addNewSensorData(SensorData data) {
        // Add new data to the sensor data list
        sensorData.add(data);
        // Notify all observers about the new data
        notifyObservers(data);
    }
}

class Dashboard implements Observer {
    void update(SensorData data) {
        newDataPoint(data);
    }

    void newDataPoint(SensorData data) {

```



```

        // Implementation for updating the dashboard with the new data point
    }
}

class StatsDisplay implements Observer {
    void update(SensorData data) {
        newMeasurement(data.getTemperature(), data.getHumidity(),
data.getWindSpeed());
    }

    void newMeasurement(double temp, double humidity, double windSpeed) {
        // Implementation for updating the statistics display
    }
}

```

Pregunta 3 (10 puntos)

Enunciado

Al visor de estadísticas del sistema meteorológico (clase *StatsDisplay*), los científicos de datos han detectado que en algunos casos necesitan la media absoluta, el máximo y el mínimo de las diferentes medidas, mientras que en otros casos necesitan la media, el máximo y mínimo de los últimos N valores. Por ejemplo, los últimos 5 valores guardados (N=5). Por su funcionamiento, la clase *StatsDisplay* tendría que almacenar (internamente en la clase o en clases asociadas) las temperaturas o valores que son necesarios para calcular la media, el máximo y el mínimo de las temperaturas.

Si tenemos los últimos 10 valores de temperatura: 10, 12, 13, 14, 15, 16, 17, 18, 19, 20, la media absoluta es 15,4 y la media relativa de los últimos N valores (cuando N es 5) es 18.

Se nos pide que implementemos una nueva versión de *StatsDisplay* que permita a los científicos de datos elegir entre valores absolutos y relativos de los últimos N valores. Los científicos también nos advierten que otros posibles cálculos pueden ser útiles en el futuro, por ejemplo, el decrecimiento exponencial.

Para simplificar, tened en cuenta que estas características no dependen de la solución del ejercicio 2. Solo tenéis que suponer que *StatsDisplay* tiene un método llamado *newMeasurement* que se tiene que llamar siempre que haya datos nuevos. Para este ejercicio solo hacen falta los valores de temperatura, se pueden ignorar los otros valores de humedad y velocidad del viento.

A continuación disponéis de un fragmento del diagrama de clases inicial con solo la clase *StatsDisplay*:

StatsDisplay

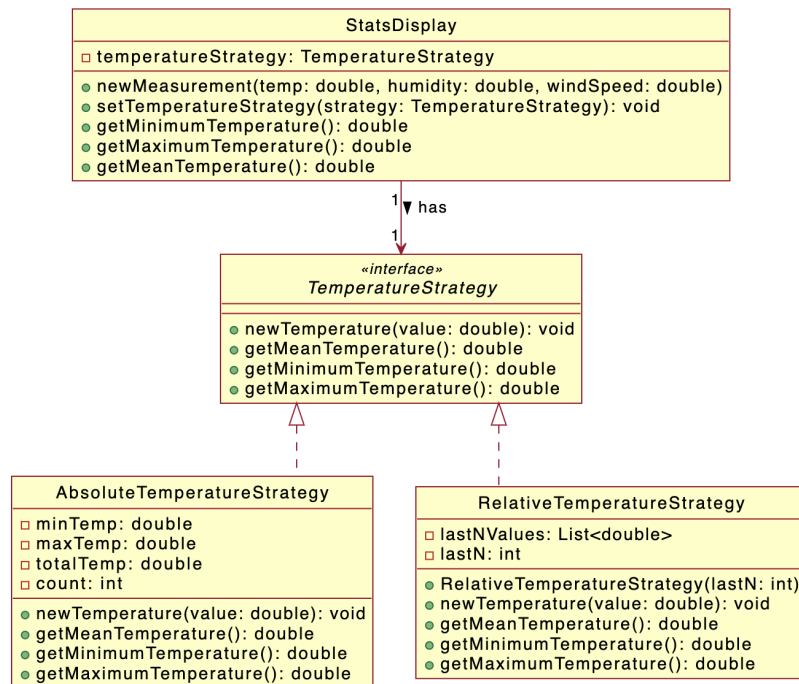
- newMeasurement(temp: double, humidity: double, windSpeed: double)
- getMinimumTemperature(): double
- getMaximumTemperature(): double
- getMeanTemperature(): double

Se pide:

- a) (3 puntos) Justifica qué patrón de diseño se tiene que aplicar para permitir, de forma flexible e intercambiable (valores absolutos y relativos), el cálculo de la media, el mínimo y el máximo de la clase *StatsDisplay*.
- b) (3 puntos) Muestra el diagrama de clases con las operaciones necesarias para implementar el diseño.
- c) (4 puntos) Muestra el pseudocódigo completo de las clases con las operaciones o el diagrama de secuencia de las operaciones *StatsDisplay::getMinimumTemperature()*, *StatsDisplay::getMaximumTemperature()* and *StatsDisplay::getMeanTemperature()* mostrando como se implementan los cálculos de los valores relativos y absolutos.

Solución

- a) El **patrón Estrategia** es ideal para este escenario. Permite que la clase *StatsDisplay* utilice diferentes algoritmos que se pueden cambiar de forma dinámica. Este patrón permite al sistema variar el algoritmo utilizado para analizar los datos meteorológicos sin cambiar los clientes que utilizan los datos. Cada algoritmo se puede encapsular dentro de su propia clase, implementando una interfaz común, que permite una fácil intercambiabilidad y extensión.
- b) A continuación disponéis del diagrama de clases resultante con la aplicación del patrón estrategia:



c) A continuació disponéis del pseudocódigo de las clases involucradas en las operaciones que se piden:

```

interface TemperatureStrategy {
    void newTemperature(double value);
    double getMeanTemperature();
    double getMinimumTemperature();
    double getMaximumTemperature();
}

class StatsDisplay {
    private TemperatureStrategy temperatureStrategy;

    public StatsDisplay(TemperatureStrategy ts) {
        this.temperatureStrategy = ts;
    }

    public void newMeasurement(double temp, double humidity, double windSpeed) {
        temperatureStrategy.newTemperature(temp);
    }

    public void setTemperatureStrategy(TemperatureStrategy strategy) {
        this.temperatureStrategy = strategy;
    }
}
    
```

```

    }

    public double getMeanTemperature() {
        return temperatureStrategy.getMeanTemperature();
    }

    public double getMinimumTemperature() {
        return temperatureStrategy.getMinimumTemperature();
    }

    public double getMaximumTemperature() {
        return temperatureStrategy.getMaximumTemperature();
    }
}

class AbsoluteTemperatureStrategy implements TemperatureStrategy {
    private double minTemp = Double.MAX_VALUE;
    private double maxTemp = Double.MIN_VALUE;

    private double sumTemp = 0;
    private double meanTemp = 0;

    private int count = 0;

    public void newTemperature(double value) {
        minTemp = Math.min(minTemp, value);
        maxTemp = Math.max(maxTemp, value);
        sumTemp += value;
        count++;
        meanTemp = sumTemp/count;
    }

    public double getMeanTemperature() {
        return meanTemp;
    }

    public double getMinimumTemperature() {
        return minTemp;
    }

    public double getMaximumTemperature() {
        return maxTemp;
    }
}

```

```

    }
}

class RelativeTemperatureStrategy implements TemperatureStrategy {
    private List<Double> lastNValues;
    private int lastN;

    public RelativeTemperatureStrategy(int lastN) {
        this.lastN = lastN;
        this.lastNValues = new ArrayList<>();
    }

    public void newTemperature(double value) {
        if (lastNValues.size() == lastN) {
            lastNValues.remove(0);
        }
        lastNValues.add(value);
    }

    public double getMeanTemperature() {
        if (lastNValues.isEmpty()) return Double.NaN;

        double sum = 0;
        for (double value : lastNValues) {
            sum += value;
        }
        return sum / lastNValues.size();
    }

    public double getMinimumTemperature() {
        if (lastNValues.isEmpty()) return Double.NaN;

        double minTemp = lastNValues.get(0);
        for (double value : lastNValues) {
            if (value < minTemp) minTemp = value;
        }
        return minTemp;
    }

    public double getMaximumTemperature() {
        if (lastNValues.isEmpty()) return Double.NaN;

```

```

    double maxTemp = lastNValues.get(0);
    for (double value : lastNValues) {
        if (value > maxTemp) maxTemp = value;
    }
    return maxTemp;
}
}

```

En este diseño, la clase *WeatherDataAnalyzer* puede utilizar diferentes algoritmos para analizar datos meteorológicos. El algoritmo específico se puede configurar en tiempo de ejecución, permitiendo estrategias de análisis flexibles y dinámicas. Este diseño hace que sea fácil añadir nuevos algoritmos de análisis sin modificar la estructura del sistema existente.

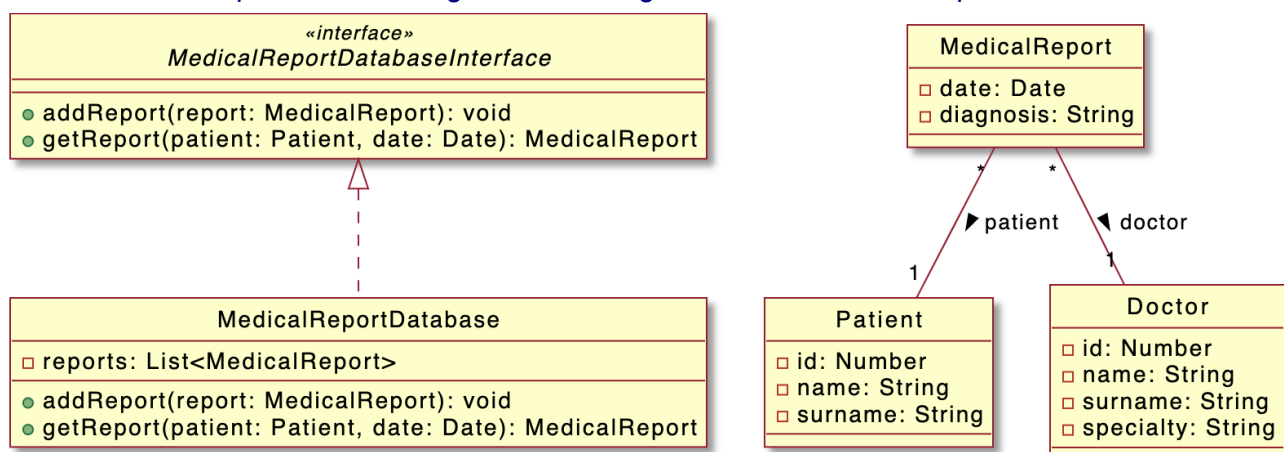
Pregunta 4 (10 puntos)

Enunciado

Un hospital ha desarrollado una solución para gestionar los informes médicos de sus pacientes. El sistema proporciona una interfaz para añadir nuevos informes y obtener informes identificados por paciente y fecha. Los informes médicos contienen la fecha, los datos del paciente y del médico, así como el diagnóstico.

Nos han contratado para integrar este sistema en el sistema de otro hospital externo.

A continuación disponéis de un fragmento del diagrama de clases del hospital:

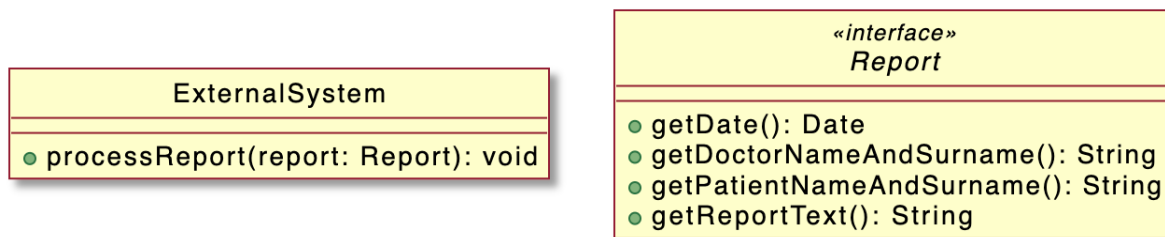


Restricciones de integridad

- *MedicalReport* se identifica por *date* y *id* del *Patient*.

- *Doctor* se identifica por su *id*.
- *Patient* se identifica por su *id*.

En el otro hospital (hospital externo) tienen un sistema para gestionar los informes médicos con la interfaz que tenéis a continuación:



Queremos que los informes médicos del primer hospital se puedan utilizar en el sistema del hospital externo de la manera más transparente y desacoplada posible.

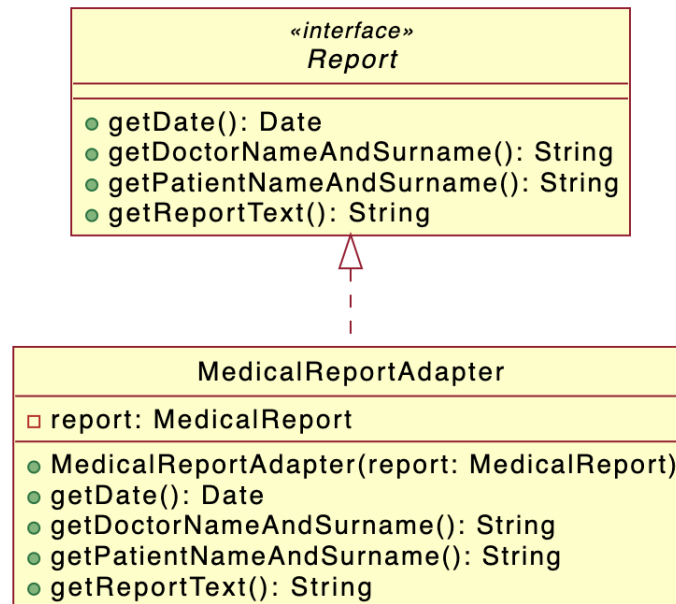
Se pide:

- (3 puntos) Justifica qué patrón de diseño se tiene que aplicar para permitir que los informes médicos existentes del primer hospital se puedan usar en el sistema del hospital externo con los mínimos cambios.
- (3 puntos) Muestra el diagrama de clases con las operaciones necesarias para implementar el diseño.
- (4 puntos) Muestra el pseudocódigo completo de las nuevas clases añadidas o de las clases existentes si hay que modificarlas. Notad que no necesitáis implementar nada de la clase *ExternalSystem*. vuestra solución sólo necesita proporcionar clases, los objetos de las cuales, puedan ser pasadas como parámetro del método *ExternalSystem::processReport()* y de este modo poder procesar todos los tipos de informes médicos existentes (de los dos hospitales).

Solución

- El **patrón Adaptador** tiene que ser utilizado para permitir que los informes médicos del primer sistema puedan ser usados en el sistema del hospital externo. El patrón permite el uso de clases existentes en el nuevo sistema sin modificarlas.
- A continuación disponéis del diagrama de clases resultante con la aplicación del patrón adaptador:

La clase *MedicalReportAdapter* implementa la interfaz *Report* y contiene un objeto de tipo *MedicalReport*.



c) A continuación disponéis del pseudocódigo de la clase proporcionada del apartado anterior:

```

class MedicalReportAdapter implements Report {
    private report: MedicalReport

    public MedicalReportAdapter(MedicalReport report) {
        this.report = report;
    }

    public Date getDate() {
        return report.getDate();
    }

    public String getDoctorNameAndSurname() {
        return report.getDoctor().getName() + " " +
report.getDoctor().getSurname();
    }

    public String getPatientNameAndSurname() {
        return report.getPatient().getName() + " " +
report.getPatient().getSurname();
    }
}
  
```



```

    public String getReportText() {
        return report.getDiagnosis();
    }
}

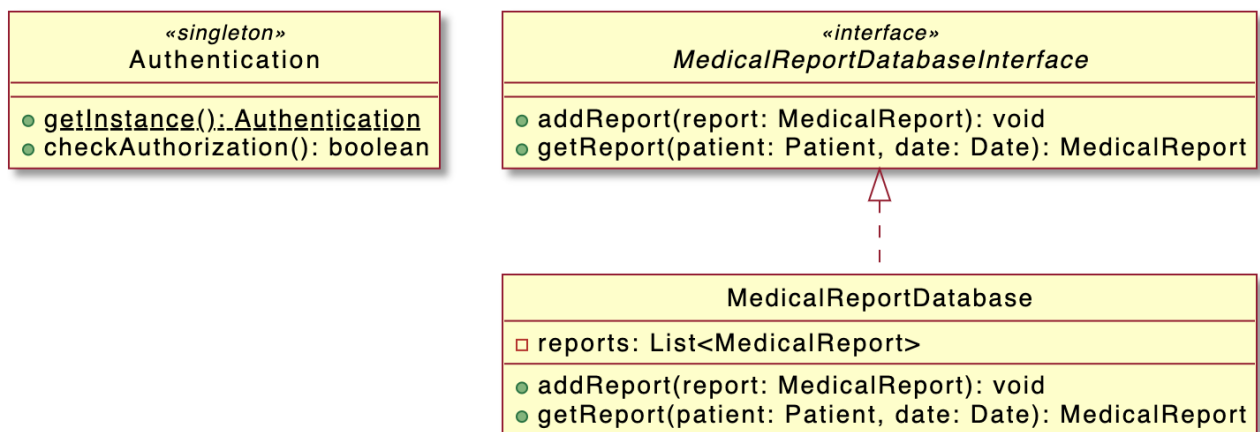
```

Pregunta 5 (5 puntos)

Enunciado

Después de resolver el problema del ejercicio anterior, el hospital quiere aumentar la seguridad de su sistema. Quieren asegurarse que solo los usuarios externos autorizados puedan acceder y añadir nuevos informes médicos.

El sistema dispone de una interfaz para acceder a la base de datos de informes médicos. Además, se dispone de una clase Singleton *Authentication* para comprobar que el usuario que intenta acceder al informe está autorizado. Esta comprobación se hace cuando se llama a la operación *Authentication::checkAuthorization()*. Si el usuario está autorizado, el método devuelve true, y devuelve false en caso contrario. Podéis suponer que el método *Authentication::checkAuthorization()* puede determinar el usuario actual por sí mismo.



Para aumentar la seguridad del sistema del hospital, vuestra solución tiene que activar una excepción cuando el usuario no esté autorizado. Queremos implementar esta comprobación de seguridad al sistema, pero queremos hacerlo de forma que se requieran los mínimos cambios y que la solución proporcionada tenga el mínimo acoplamiento posible.

Se pide:

- (1 punto) Justifica qué patrón de diseño se tiene que aplicar para gestionar la seguridad de la base de datos de informes médicos.

- b) (2 puntos) Muestra el diagrama de clases con las operaciones necesarias para implementar el diseño.
- c) (2 puntos) Muestra el pseudocódigo completo de tu solución.

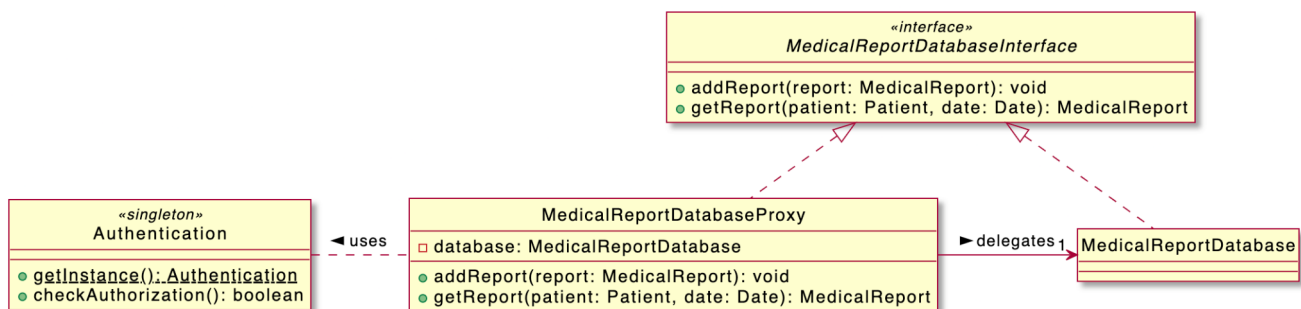
Solución

- a) El patrón de diseño más adecuado para implementar este escenario es el **patrón Representante (Proxy)** puesto que:

Minimiza el acoplamiento: El patrón Proxy permite al sistema desacoplar la comprobación de seguridad, de las operaciones reales de la base de datos, cosa que es esencial para la extensibilidad del sistema.

Incrementa la seguridad y transparencia del sistema: El patrón Proxy permite al sistema implementar comprobaciones de seguridad sin modificar las operaciones existentes de la base de datos, cosa que es esencial para la integridad y transparencia del sistema.

- b) A continuación disponéis del diagrama de clases resultante con la aplicación del patrón Representante:



- c) A continuación disponéis del pseudocódigo de la clase Representante del apartado anterior:

```

class MedicalReportDatabaseProxy implements MedicalReportDatabaseInterface {
    private MedicalReportDatabase database;

    public MedicalReportDatabaseProxy(MedicalReportDatabase database) {
        this.database = database;
    }

    public void addReport(MedicalReport report) {
        if(Authentication.getInstance().checkAuthorization()==false) {
            throw new AuthorizationException();
        }
    }
}
    
```

```
    }  
    database.addReport(report);  
}  
  
public MedicalReport getReport(Patient patient, Date date) {  
    if(Authentication.getInstance().checkAuthorization()==false) {  
        throw new AuthorizationException();  
    }  
    return database.getReport(patient, date);  
}  
}
```